

Experiment Report 1

Diego Manuel Maldonado Núñez
Student ID: 202620726

May 7, 2026

1 Introduction

Large language model assistants have become a common tool in software development, yet their effect on individual coding habits remains largely anecdotal. This experiment takes a personal, data-driven approach to that question: how did my own coding habits change after I began relying on these tools?

According to my records, I started using large language model assistants on June 9, 2025. Using that date as a natural intervention point, I compare two equal-duration periods — before and after adoption — across two dependent variables: merge requests per month, which could include both feature and bugfix completion, and active days per month, which captures coding frequency. By analyzing these metrics, I aim to uncover any shifts in my coding patterns that may be attributable to the use of large language model assistants. measured as merge commits per month, and coding frequency, measured as active days per month.

2 Method

In short, I will perform an Observational Experiment, collecting data on my coding activity across GitHub and GitLab, and then aggregate it by month to analyze it descriptively. With this, we will compute point estimators (sample means and sample standard deviations of 1 km time for each drink condition) and interval estimators (95 confidence intervals for the mean times using Student's t-distribution, because our sample size is small, due to sampling the data by month).

The independent variable is the use of large language model assistants, which is considered a binary variable with two levels: before and after I started using them. The start date of using large language model assistants is June 9, 2025, according to my records.

The dependent variables are the following:

- Merge commits per month, which captures feature completion rate/bugfix completion rate. While not all merge commits necessarily correspond to completed features or bugfixes, in my workflow they typically represent the culmination of a coding effort on my part.
- Active days per month on the platform per month, which captures coding frequency. While there is a possibility of doing coding sessions without committing, in my workflow I always commit at the end of a coding session, to make sure I do not lose work and to keep track of my progress.

2.1 Data Collection Protocol

This repository collects statistics from my git activity and stores them in a CSV file. The code for generating this dataset is available in the repository, and it iterates over a certain folder looking for git repositories. For each repository, it iterates over the commit data. How this can be run is located in the README.md file. It is a uv command. Each observation corresponds to a commit and includes the commit date, commit hash, total lines added and removed, and the total lines changed per file. For privacy reasons, the dataset does not include the repository name or the file names, but it does include the programming language of each file, and the amount of lines changed per file, and programming language. It also only includes commits of repositories which were created by me and are not under a Non-Disclosure Agreement (NDA). The fact that some repositories are not included in the dataset is a potential source of bias. However, the amount of repositories that are not included is small, and they are projects which I have not been actively working on during the period of analysis, so I do not expect them to have a significant impact on the results.

2.2 Data Analysis Protocol

I performed an exploratory data analysis (EDA) on the gathered dataset. This EDA can also be found in the repository.

I first loaded the CSV, inspected the data, and then cleaned it. The cleaning steps were:

1. Deduplicate by commit hash, keeping one row per commit.
2. Remove rows sharing the same per-language line profile with a different hash or date, to avoid inflated counts from repeated entries.
3. Exclude languages not directly written by me (e.g. auto-generated code from Protocol Buffers/Thrift, third-party front-end libraries, Jupyter Notebook output) and subtract generated Python/Go lines from commits that also contain Protocol Buffers or Thrift files.

4. Remove commit-level outliers with more than 3,000 lines added, which likely correspond to library imports rather than original code.

To perform the EDA, I focused on a time period which is equal in duration before and after I started using large language model assistants. I chose this approach to avoid the bias that would come from comparing a much longer time period before I started using assistants against a much shorter period after. For this reason, the selected time period goes from September 2024 to April 2026, which includes the same number of complete calendar months before and after I started using large language model assistants in June 2025.

3 Result

3.1 Merge Commits

The first metric is the number of merge commits per month. I interpret this as the most important, given that I usually create merge requests, and therefore merge commits, when I finish implementing a certain feature.

Period	Mean/month	Median/month	Std./month	Coef. Variation	95% CI
Before	14.20	13.50	4.59	0.32	[10.9, 17.5]
After	14.50	13.00	6.17	0.43	[10.1, 18.9]

Table 1: Descriptive statistics for monthly merge commit counts in the equal-duration before/after window. The 95% CI for the mean uses the t -distribution ($n = 10$).

The two confidence intervals overlap substantially: the overlap region is [10.9, 17.5], a width of 6.6 — equal to the entire before-period interval. This means the true means for both periods could plausibly be the same value, and the CIs alone provide no evidence of a difference.

The coefficients of variation are somewhat high, especially for the After period, might indicate that the mean is not a good point estimator for the tendency of the data, and this might also make the confidence intervals less informative. The median might be a better point estimator in this case, as it is more robust to variability and outliers.

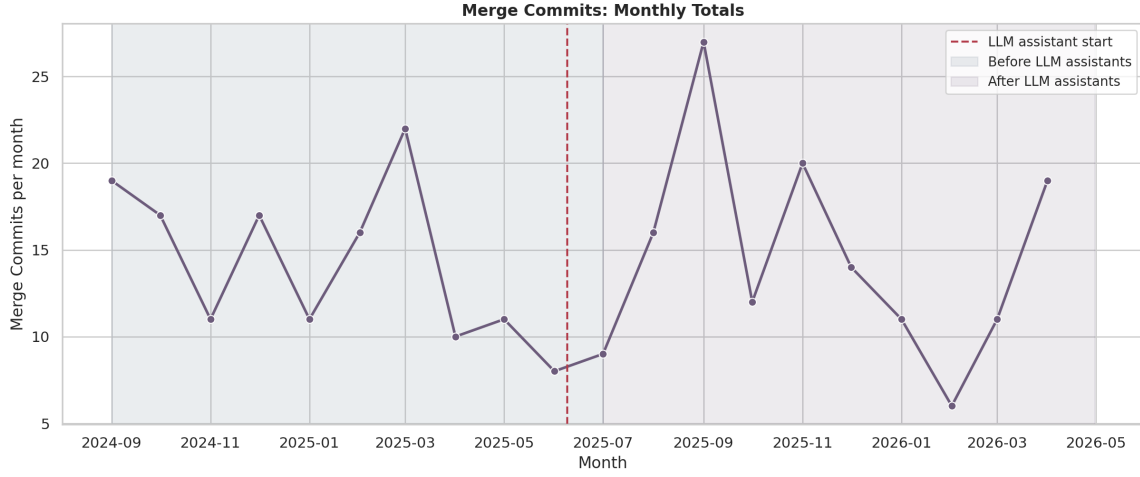


Figure 1: Monthly merge commit counts over time.

By taking a look at this plot, it can be seen that while it seems that there are higher peaks after the start of using large language model assistants, the lows are also below what was observed before, therefore it can not be concluded that there was a clear increase in merge commits without looking at the total amounts.

Period	Start	End	Months	Total merge commits
Before	2024-09-01	2025-06-30	10	142
After	2025-07-01	2026-04-30	10	145

Table 2: Equal-duration window total merge commits using the same number of calendar months before and after the large-language-model-assistant start date.

Looking at this table, the monthly merge commit totals are very similar across both periods. This indicates that I have been completing a very similar number of features independently of the use of large language model assistants.

3.2 Active Days per Month

For the second variable, I looked at the number of active days per month, to determine if there has been a change in this by introducing usage of large language models.

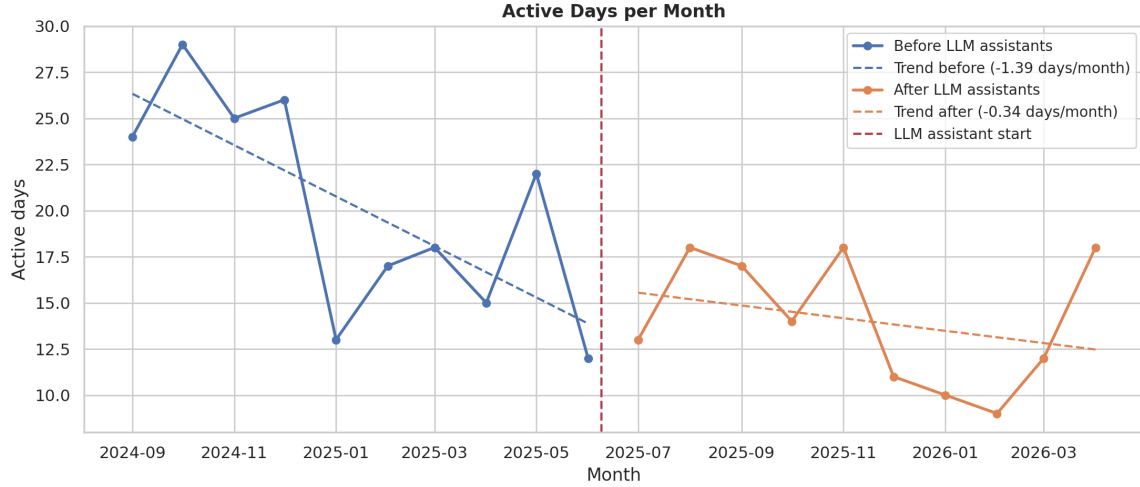


Figure 2: Active days per month.

Period	Months	Total	Mean	Std.	Min	Max	Coef. Variation	95% CI
Before	10	201	20.1	5.9	12	29	0.29	[15.9, 24.3]
After	10	140	14.0	3.5	9	18	0.25	[11.5, 16.5]

Table 3: Descriptive statistics for active days per month. The 95% CI for the mean uses the t -distribution ($n = 10$).

The two confidence intervals barely overlap: the overlap region is $[15.9, 16.5]$, a width of only 0.6. This is a practically and statistically meaningful difference. Since the mean active days per month decreased from 20.1 to 14.0, this suggests a significant reduction in coding frequency after starting to use large language model assistants. In this case, the coefficients of variation are somewhat lower, which might indicate that the mean is a better point estimator in this case, but the variability is still somewhat high, and the median might also be a better point estimator.

With that in mind, when analyzing results for both variables together, it seems that while the amount of work I do in terms of merge commits has not changed significantly, the frequency with which I code has decreased. This could suggest that I am accomplishing a similar amount of work in fewer coding sessions.

4 Conclusion

Looking at the results, total code production stayed roughly for merge commits. Yet at the same time, the number of days per month on which I actively coded fell in the same period.

These two observations together suggest that large language model assistants have helped me accomplish a similar amount of work in fewer coding sessions. Rather than coding more often or more intensely, I appear to be coding less frequently while still producing a comparable output.

One thing I did notice through the experiment, is that the mean as a point estimator is not a good estimator, as the coefficients of variation for both dependent variables are somewhat high. This is also backed up by the fact that the amount of work generally is highly variable across months, and using the mean is not a good way to capture the tendency of the data. The confidence interval could replace the mean with the median to overcome this as well. However, due to the time constraints of the experiment, I did not have time to compute confidence intervals for the median. This is something that could be explored in future work.